A Systematic Benchmark of Persistent Homology Pipelines for Classification

Zachariah Attila Markusson

August 2026

Independent Research

Abstract

Persistent homology classification pipelines have three stages — filtration, vectorization, and classifier — each with multiple available methods. Which stage matters most? Benchmarking all three stages simultaneously in a controlled factorial sweep — 616 configurations across 6 dataset instances, 4 filtrations, 7 vectorizers, and 4 classifiers (56 runs failed where point-cloud filtrations are incompatible with image data) — we find that the vectorization stage dominates classification-accuracy variance on both real datasets: on ECG200 time series (Takens-embedded) it has a marginal range of 6.09pp across 7 vectorizers (95% CI [5.84, 6.34]; $\eta^2 = 0.217$) versus 0.70pp for filtration (95% CI [0.35, 1.04]), and on binary MNIST it again leads (3.22pp; $\eta^2 = 0.302$) while the filtration effect is modest (cubical 98.0% vs. Vietoris–Rips 96.25% best of family, 1.75pp). A second time series, ECG5000, replicates the hierarchy (24.89pp vs. 3.60pp). These results complement, rather than refute, the image-data case study of Conti et al. [9], whose 18%–94% range spans a joint filtration-and-vectorization redesign (their H0-only naive cubical-binary pipeline to improved-filtration multivectors), part of which is attributable to the vectorization stage (their 18% figure is the H0-only naive cubical-binary pipeline; the 94% figure adds improved filtrations on 10-class MNIST). On scale-matched genus-1 versus genus-2 synthetic point clouds, norm features classify at chance (48–58%) while the TDA pipeline sustains 95.83% accuracy at $\sigma = 0.30$ — the noise robustness is not an artefact of the norm/scale confound. Measured bottleneck distances (max $d_B = 0.434$ vs. the corrected bound $2\sigma\sqrt{2\ln n} \approx 1.82$ at $\sigma = 0.30$) confirm the stability bound is conservative. The framework is YAML-configurable and extensible to 7 filtrations, 11 vectorizers, and 4 classifiers; all code, configuration, and result schemas are provided.

Contents

List of Figures

List of Tables

Chapter 1

Introduction and Motivation

1.1 Topological Data Analysis and the Pipeline Problem

Topological Data Analysis (TDA) extracts shape invariants from data: connected components, cycles, and voids that persist across multiple scales. Unlike traditional machine learning which operates on raw coordinates or pixel intensities, TDA captures the underlying "shape" of the data — information invariant under stretching and bending. Persistent homology, the workhorse of TDA, tracks the birth and death of topological features as a scale parameter varies.

Practitioners face a combinatorial problem: the TDA classification pipeline has three stages — filtration, vectorization, and classification — each with 5–15 available methods (surveys: Ali et al. [2] and Hensel et al. [24]). The literature's most cited answer to "which stage matters most?" comes from Conti et al. [9], who showed that filtration choice on MNIST images can be catastrophic: accuracy ranged from 18% (their cubical pipeline) to 94% (improved filtrations), under 10-split cross-validation with grid search. That study has sometimes been read as a general principle — "filtration dominates" — although the authors themselves did not claim one.

This paper asks the same question across two real datasets of different modalities and finds that vectorization is the dominant stage on both: ECG200 time series and binary MNIST. On ECG200, vectorization moves accuracy by 6.09pp (95% CI [5.84, 6.34]) versus 0.70pp for filtration (95% CI [0.35, 1.04]); on binary MNIST the vectorizer range (3.22pp) is twice the filtration range (1.65pp), and cubical beats Vietoris-Rips by ∼1.75pp best-of-family. Filtration effects are dataset-specific and modest on binary MNIST, not the governing stage. We demonstrate this using a modular benchmarking framework that varies all three pipeline stages simultaneously in a controlled factorial sweep: 6 dataset instances × 4 filtrations (Vietoris-Rips, weak Alpha, Sparse Rips, Cubical) × 7 vectorizers × 4 classifiers — 672 possible, 616 completed (56 runs failed

(`weak_alpha`/`sparse_rips` on MNIST) and were excluded). The framework is extensible to 7 filtrations, 11 vectorizers, and 4 classifiers via YAML-only configuration changes; standardised harnesses of this kind are the direction the community has moved towards [19].

1.2 The Persistent Homology Pipeline

Whether persistent homology features help classification at all has itself been questioned [20]; this benchmark assumes the features are informative and asks which pipeline stage extracts them most effectively.

A persistent homology classification pipeline transforms raw data through three stages:

1. **Filtration.** Raw data is converted into a nested sequence of simplicial complexes parameterised by a scale ε. Topological features appear (birth at b) and disappear (death at d). Output: a *persistence diagram*, a multiset of (b, d) pairs.

2. **Vectorization.** Diagrams are multisets of variable cardinality and cannot be fed directly into classifiers. Vectorization methods map each diagram to a fixed-length feature vector.

3. **Classification.** The vectorized features are passed to a standard classifier which learns a decision boundary.

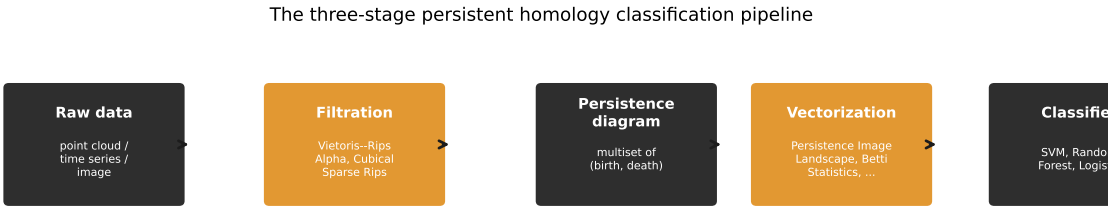

Figure 1.1: The three-stage persistent homology classification pipeline.

1.3 Contributions

This paper contributes a reproducible factorial benchmark framework and two case studies that use it to locate the dominant pipeline stage. Specifically:

1. **Framework.** A benchmarking framework that varies all three pipeline stages simultaneously: 4 filtrations, 7 vectorizers, and 4 classifiers executed in a 616-configuration factorial sweep (of 672 possible; 56 runs failed (`weak_alpha`/`sparse_`

`rips` on MNIST) and were excluded), with YAML-driven configuration and normalised SQLite storage. The framework is extensible to 7 filtrations, 11 vectorizers, and 4 classifiers via YAML-only changes. Full configuration and schema in Appendices A–B; code implementations in Appendix C.

2. **Verified finding: vectorization dominates.** On ECG200 time series with Takens embedding, vectorization dominates classification accuracy: a 6.09pp marginal range across 7 vectorizers (95% CI [5.84, 6.34]), versus 0.70pp across 4 filtrations (95% CI [0.35, 1.04]; small but statistically non-zero) and 3.18pp across 4 classifiers (95% CI [2.57, 3.78]), with the ordering vectorizer > classifier > filtration stable across all five cross-validation repetitions. On binary MNIST the vectorizer again dominates (marginal range 3.22pp versus 1.65pp for filtration), and cubical beats Vietoris-Rips by ∼1.75pp best-of-family (98.0% vs. 96.25%); a second time-series dataset (ECG5000) replicates the hierarchy (24.89pp versus 3.60pp). Filtration effects are dataset-specific and modest on binary MNIST, not the governing stage. Conti et al.'s MNIST case study (18% with their cubical pipeline to 94% with improved filtrations, 10-class, grid search) is a complementary demonstration that a badly-matched filtration can be catastrophic on images; it does not establish a general law.

3. **Matched-scale synthetic evidence.** On synthetic genus-1 versus genus-2 point clouds whose radial norm distributions are quantile-matched, norm features classify at chance (48–58%) while the TDA pipeline sustains 95.83% accuracy at $\sigma = 0.30$ noise — the noise robustness is not an artefact of the norm/scale confound between the sphere and torus geometries.

4. **Stability-bound confirmation.** Measured bottleneck distances between clean and noisy diagrams (max $d_B = 0.434$ over 80 cloud/dimension pairs) sit well below the corrected worst-case bound $d_B \leq 2\sigma\sqrt{2 \ln n}$ (≈ 1.82 at $\sigma = 0.30$, $n = 100$), confirming the bound is conservative in practice.

Chapter 2 develops the mathematical machinery needed to understand why these pipeline stages interact as they do.

Chapter 2

Mathematical Foundations of Persistent Topology

This chapter develops the algebraic and geometric machinery underlying persistent homology. We progress from simplicial complexes through chain groups, homology vector spaces, filtrations, and persistence modules, concluding with the stability theorem and vectorization methods that convert diagrams into classifier-ready feature vectors.

2.1 Simplicial Complexes

Definition 2.1 (Simplex)**.** A *k-simplex* $\sigma = [v_0, v_1, \ldots, v_k]$ is the convex hull of $k+1$ affinely independent points in $\mathbb{R}^d$ ($d \geq k$). A 0-simplex is a vertex, a 1-simplex an edge, a 2-simplex a triangle, a 3-simplex a tetrahedron. A *face* of σ is the convex hull of any non-empty subset of its vertices.

Definition 2.2 (Simplicial Complex)**.** A *simplicial complex* K is a finite collection of simplices such that:

1. If $\sigma \in K$ and τ is a face of σ, then $\tau \in K$ (closure under taking faces).
2. If $\sigma, \tau \in K$, then $\sigma \cap \tau$ is either empty or a face of both.

The *k-skeleton* $K^{(k)}$ is the subcomplex of all simplices of dimension at most k. The 1-skeleton is a graph in the usual sense.

2.2 Filtrations and Complex Constructions

Definition 2.3 (Filtration)**.** A *filtration* of a simplicial complex K is a nested sequence of subcomplexes indexed by a real parameter $\varepsilon \geq 0$:

$$\emptyset = K_{\varepsilon_0} \subseteq K_{\varepsilon_1} \subseteq \cdots \subseteq K_{\varepsilon_m} = K, \tag{2.1}$$

where $\varepsilon_0 < \varepsilon_1 < \cdots < \varepsilon_m$.

Three filtrations are used in the benchmark:

Vietoris-Rips complex $\mathrm{VR}_\varepsilon(X)$. For a finite point cloud $X \subset \mathbb{R}^d$, a k-simplex $[x_0, \ldots, x_k]$ is included if $\|x_i - x_j\| \leq \varepsilon$ for all $0 \leq i < j \leq k$. The Rips complex is simple to compute but large: for n points, the number of k-simplices scales as $O(n^{k+1})$. Specifically, the number of 2-simplices (triangles) in the full Rips complex is $O(n^3)$, which drives the runtime differences observed in Chapter 4.

Alpha complex $A_\varepsilon(X)$. Constructed via the intersection of Voronoi cells with metric balls. For each point $x_i \in X$, let $V_i = \{y \in \mathbb{R}^d : \|y - x_i\| \leq \|y - x_j\| \text{ for all } j \neq i\}$ be its closed Voronoi cell, and define $R_i(\varepsilon) = V_i \cap \overline{B}_\varepsilon(x_i)$ where $\overline{B}_\varepsilon(x_i)$ is the closed ball of radius $\varepsilon/2$. The Alpha complex is the nerve of the cover $\{R_i(\varepsilon)\}_{i=1}^n$. The Nerve Theorem (see below) guarantees homotopy equivalence to the union of balls.

Theorem 2.1 (Nerve Theorem — Leray 1945 [12])**.** *Let $\mathcal{U} = \{U_i\}_{i \in I}$ be a finite open cover of a topological space M such that every non-empty intersection $\bigcap_{j \in J} U_j$ is contractible. Then the nerve $N(\mathcal{U})$ is homotopy equivalent to $\bigcup_{i \in I} U_i$.*

For the Alpha cover $\{R_i(\varepsilon)\}$, the sets $R_i(\varepsilon)$ are convex polyhedra in $\mathbb{R}^d$, so all finite intersections are convex and thus contractible. The Nerve Theorem therefore guarantees that $A_\varepsilon(X) \simeq \bigcup_i \overline{B}_{\varepsilon/2}(x_i)$. In $\mathbb{R}^3$, the Alpha complex is a subcomplex of the Delaunay triangulation with $O(n^2)$ simplices, compared to $O(n^3)$ for the full Rips complex.

Remark 2.1. The benchmark executes giotto-tda's `WeakAlphaPersistence`, which per giotto's documentation is the Vietoris–Rips filtration of the sparse matrix of distances between Delaunay-neighbouring vertices — a subcomplex of the full Rips complex (cheaper to build), not a subcomplex of the Alpha complex, and not covered by the Nerve-Theorem homotopy guarantee, which applies to the full Alpha/Čech complex. For the executed approximation we verify empirically in §4.3 that the full GUDHI Alpha complex (persistence-equivalent to the Čech complex) reproduces weak-alpha classification accuracy (mean difference 0.0–0.8pp) at 2–5× the runtime cost, so the approximation is faithful on these tasks.

Sparse Rips complex. A subsampled approximation of VR that retains only a sparse subset of edges and higher simplices, designed for point clouds with $n \geq 10^3$ points where full VR computation is prohibitive. The sparsification parameter ϵ controls the approximation fidelity.

2.3 Homology Groups and Persistent Homology

Chain Complexes and Homology

To define persistent homology formally, we first construct the algebraic machinery of simplicial homology over the field $\mathbb{Z}_2 = \mathbb{Z}/2\mathbb{Z}$.

Definition 2.4 (Chain Group)**.** For a simplicial complex K, the *k-th chain group* $C_k(K; \mathbb{Z}_2)$ is the $\mathbb{Z}_2$-vector space whose basis elements are the k-simplices of K. An element $c \in C_k(K; \mathbb{Z}_2)$ is a formal sum $c = \sum_{\sigma \in K^{(k)}} a_\sigma \sigma$ with coefficients $a_\sigma \in \mathbb{Z}_2$.

Definition 2.5 (Boundary Operator)**.** The *k-th boundary operator* $\partial_k : C_k(K; \mathbb{Z}_2) \to C_{k-1}(K; \mathbb{Z}_2)$ is the linear map defined on a k-simplex $\sigma = [v_0, \ldots, v_k]$ by:

$$\partial_k [v_0, \ldots, v_k] = \sum_{i=0}^{k} [v_0, \ldots, \hat{v}_i, \ldots, v_k], \tag{2.2}$$

where $\hat{v}_i$ denotes omission of v_i, and the sum is taken over $\mathbb{Z}_2$ (so $1+1=0$). For $k = 0$, $\partial_0 = 0$ by convention.

The fundamental property of the boundary operator is $\partial_k \circ \partial_{k+1} = 0$ for all $k \geq 0$. Intuitively, the boundary of a boundary is empty: the edges bounding a triangle form a closed loop with no endpoints.

Definition 2.6 (Homology Group)**.** For a simplicial complex K, define:

- $Z_k(K) = \ker \partial_k$ — the *k-cycles* (chains with zero boundary).
- $B_k(K) = \operatorname{im} \partial_{k+1}$ — the *k-boundaries* (chains that are boundaries of $(k+1)$-chains).

Since $\partial_k \circ \partial_{k+1} = 0$, we have $B_k(K) \subseteq Z_k(K)$. The *k-th homology group* is the quotient vector space:

$$H_k(K) = \frac{Z_k(K)}{B_k(K)} = \frac{\ker \partial_k}{\operatorname{im} \partial_{k+1}}. \tag{2.3}$$

Elements of $H_k(K)$ are equivalence classes of k-cycles, where two cycles are equivalent if they differ by a boundary. The dimension of $H_k(K)$ as a $\mathbb{Z}_2$-vector space is the *k-th Betti number*: $\beta_k(K) = \dim_{\mathbb{Z}_2} H_k(K)$. Informally, β_0 counts connected components, β_1 counts 1-dimensional holes (cycles), β_2 counts 2-dimensional voids.

Persistent Homology

A filtration $\{K_\varepsilon\}_{\varepsilon \geq 0}$ induces a sequence of inclusion maps $i^{\varepsilon,\varepsilon'} : K_\varepsilon \hookrightarrow K_{\varepsilon'}$ for $\varepsilon \leq \varepsilon'$, which in turn induce linear maps on homology:

$$f_k^{\varepsilon,\varepsilon'} : H_k(K_\varepsilon) \to H_k(K_{\varepsilon'}). \tag{2.4}$$

Definition 2.7 (Persistent Homology Group)**.** The *k-th persistent homology group* from ε to ε' is:

$$H_k^{\varepsilon,\varepsilon'}(K) = \operatorname{im} f_k^{\varepsilon,\varepsilon'}. \tag{2.5}$$

A homology class $\gamma \in H_k(K_{\mathrm{b}})$ is *born* at $\varepsilon = \mathrm{b}$ if $\gamma \notin \operatorname{im} f_k^{\mathrm{b}-\delta,\mathrm{b}}$ for any $\delta > 0$, and *dies* at $\varepsilon = \mathrm{d} > \mathrm{b}$ if $f_k^{\mathrm{b},\mathrm{d}}(\gamma) \in \operatorname{im} f_k^{\mathrm{b}-\delta,\mathrm{d}}$ for some $\delta > 0$ but $f_k^{\mathrm{b},\mathrm{d}-\delta}(\gamma) \notin \operatorname{im} f_k^{\mathrm{b}-\delta,\mathrm{d}-\delta}$. If γ never dies, we set $\mathrm{d} = \infty$.

Definition 2.8 (Persistence Diagram)**.** The *persistence diagram* $\mathrm{Dgm}_k(K)$ is the multiset of points $(\mathrm{b},\mathrm{d}) \in \mathbb{R}^2 \cup \{\infty\}$ where $\mathrm{b} < \mathrm{d}$, together with infinitely many copies of each point on the diagonal $\Delta = \{(x,x) : x \geq 0\}$. The *persistence* of a feature is $\mathrm{p} = \mathrm{d} - \mathrm{b}$.

Example. A sphere S^2 has $\beta = (1,0,1)$: one component, no 1-cycles, one 2-void. A torus T^2 has $\beta = (1,2,1)$: one component, two independent 1-cycles (the meridional and longitudinal loops), one 2-void. This $\beta_1 = 0$ versus $\beta_1 = 2$ difference is the topological signal driving the sphere-versus-torus classification in Chapter 4.

2.4 Stability of Persistence Diagrams

The stability theorem guarantees that small perturbations of the input data produce proportionally small changes in the persistence diagram — essential for any pipeline handling noisy data.

Definition 2.9 (Bottleneck Distance)**.** The *bottleneck distance* between two persistence diagrams D_1 and D_2 is:

$$d_B(D_1, D_2) = \inf_{\gamma : D_1 \to D_2} \sup_{p \in D_1} \|p - \gamma(p)\|_\infty, \tag{2.6}$$

where γ ranges over all bijections from D_1 to D_2 (including pairings with the diagonal Δ), and $\|\cdot\|_\infty$ is the L^∞ norm on $\mathbb{R}^2$.

Definition 2.10 (Gromov-Hausdorff Distance)**.** For compact metric spaces X and Y, the *Gromov-Hausdorff distance* is:

$$d_{GH}(X,Y) = \frac{1}{2} \inf_{Z,f,g} d_H^Z(f(X), g(Y)), \tag{2.7}$$

where the infimum is taken over all metric spaces Z and all isometric embeddings $f : X \hookrightarrow Z$, $g : Y \hookrightarrow Z$, and d_H^Z is the Hausdorff distance in Z: $d_H^Z(A,B) = \max\{\sup_{a\in A} \inf_{b\in B} d_Z(a,b),\ \sup_{b\in B} \inf_{a\in A} d_Z(a,b)\}$.

Theorem 2.2 (Stability Theorem for geometric complexes — Chazal, de Silva & Oudot [23]). *Let X, Y be totally bounded metric spaces. Then:*

$$d_B(\mathrm{Dgm}_k(X), \mathrm{Dgm}_k(Y)) \leq 2\, d_{GH}(X, Y) \tag{2.8}$$

for all $k \geq 0$, where $\mathrm{Dgm}_k(\cdot)$ denotes the k-dimensional persistence diagram of the Vietoris-Rips or Čech filtration (Chazal, de Silva & Oudot 2014, Thm. 5.2). The function-level stability bound $d_B \leq \|f - g\|_\infty$ of Cohen-Steiner, Edelsbrunner & Harer [8] applies instead to the sublevel-set (cubical) filtration on grids.

This theorem is applied in §4.2: for n points with additive Gaussian noise $\eta_i \sim \mathcal{N}(0, \sigma^2 I_3)$, extreme value theory gives the typical scale of the maximum perturbation as $\mathbb{E}[\max_i \|\eta_i\|] \approx \sigma\sqrt{2 \ln n}$ (the one-dimensional leading-order value; for the executed $d = 3$ per-coordinate noise the expectation is larger by the dimension correction, $\approx 1.16\,\sigma\sqrt{2 \ln n}$ at $n = 100$; Monte Carlo gives $\mathbb{E}[\max_i \|\eta_i\|] \approx 1.06$ at $\sigma = 0.30$). The noisy cloud therefore lies within Gromov–Hausdorff distance of order $\sigma\sqrt{\ln n}$ of the clean one — a typical-case estimate, not an almost-sure bound: the threshold $\sigma\sqrt{2 \ln n}$ is exceeded by the maximum with probability $\approx 93\%$ in the executed d=3 noise (the 1-D value would be $\approx 22\%$). By Theorem 2.2 the bottleneck distance between clean and noisy diagrams satisfies

$$d_B\big(\mathrm{Dgm}_k(X),\, \mathrm{Dgm}_k(X + \eta)\big) \;\lesssim\; 2\,\sigma\sqrt{2 \ln n}. \tag{2.9}$$

The factor 2 inherited from the stability theorem is essential: for $n = 100$ (points per cloud after subsampling), $2\,\sigma\sqrt{2 \ln 100} \approx 0.91$ at $\sigma = 0.15$ and ≈ 1.82 at $\sigma = 0.30$ — a factor of two larger than the bound one would write down by omitting it. Measured bottleneck distances between clean and noisy diagrams (ripser diagrams, GUDHI bottleneck implementation; 80 cloud/dimension pairs; §4.2) are far below this: at $\sigma = 0.15$ the mean is 0.144 and the maximum 0.300; at $\sigma = 0.30$ the maximum is 0.434. The torus H_1 diagrams move by d_B on average 0.283 at $\sigma = 0.30$ — below the measured H_1 persistence range $[0.608,\ 1.156]$ — so the features comfortably survive the perturbation.

2.5 Vectorization Methods

Vectorization converts a persistence diagram into a fixed-length feature vector for classification. We use the taxonomy of Ali et al. [2]; comparative studies include Barnes et al. [3], Perea et al. [13], and Sulowska [16].

Persistence diagrams are multisets of variable cardinality. To use them with standard classifiers, we must map each diagram to a fixed-length vector in a normed space.

We present the seven methods executed in the sweep: the four classical representations below, followed by three widely used alternatives (Silhouette, Amplitude, and Persistence Entropy).

Persistence Statistics. For each homology dimension k, compute the mean, standard deviation, minimum, and maximum of births, deaths, and lifespans ($\mathrm{p} = \mathrm{d} - \mathrm{b}$) across all points in Dgm_k. This yields $4 \times 3 = 12$ features per dimension. Despite zero hyperparameters, this method is competitive with kernel-based approaches [2].

Betti Curves [7]. The Betti curve $\beta_k(\varepsilon)$ counts the number of k-dimensional features with $\mathrm{b} \le \varepsilon < \mathrm{d}$. Evaluated at n_{bins} equally spaced points in $[\varepsilon_{\min}, \varepsilon_{\max}]$, this produces n_{bins} features per homology dimension.

Persistence Landscapes [4]. For each point $(\mathrm{b}, \mathrm{d}) \in \mathrm{Dgm}_k$, define the tent function:

$$\Lambda_{(\mathrm{b},\mathrm{d})}(t) = \max\big(0,\, \min(t - \mathrm{b},\, \mathrm{d} - t)\big). \tag{2.10}$$

The *m-th persistence landscape* is the function $\lambda_m : \mathbb{R} \to \mathbb{R}$ given by:

$$\lambda_m(t) = m\text{-}\max_{p \in \mathrm{Dgm}_k} \Lambda_p(t), \tag{2.11}$$

where m-max denotes the m-th largest value. Each λ_m is 1-Lipschitz. The sequence $(\lambda_m)_{m \in \mathbb{N}}$ resides in the Banach space $L^p(\mathbb{N} \times \mathbb{R})$ for any $1 \le p \le \infty$, enabling classical statistical operations (means, variances, confidence intervals) directly on the vectorized representation. With n_{layers} layers evaluated at n_{bins} points, this produces $n_{\mathrm{layers}} \times n_{\mathrm{bins}}$ features.

Persistence Images [1]. Transform each point $(\mathrm{b}, \mathrm{d}) \in \mathrm{Dgm}_k$ to birth-persistence coordinates (b, p) where $\mathrm{p} = \mathrm{d} - \mathrm{b}$. Define a weighting function $w : \mathbb{R} \times \mathbb{R}_{\ge 0} \to \mathbb{R}_{\ge 0}$, typically:

$$w(\mathrm{b}, \mathrm{p}) = \begin{cases} \mathrm{p}/\mathrm{p}_{\max}, & \mathrm{p} \le \mathrm{p}_{\max}, \\ 1, & \text{otherwise}, \end{cases} \tag{2.12}$$

where $\mathrm{p}_{\max}$ is a fixed constant (set to the 95th percentile of persistences in our implementation). The persistence surface is:

$$\rho(x, y) = \sum_{(\mathrm{b},\mathrm{p}) \in T(\mathrm{Dgm}_k)} w(\mathrm{b}, \mathrm{p})\, \phi_{(\mathrm{b},\mathrm{p})}(x, y), \tag{2.13}$$

where $\phi_{(\mathrm{b},\mathrm{p})}(x, y) = \frac{1}{2\pi\sigma^2} \exp\!\big(-\frac{(x-\mathrm{b})^2 + (y-\mathrm{p})^2}{2\sigma^2}\big)$ is a 2D Gaussian kernel. Evaluating ρ on an $n_{\mathrm{bins}} \times n_{\mathrm{bins}}$ grid yields n_{bins}^2 features per homology dimension.

Silhouette [6]. The silhouette of a diagram weights the landscape tent functions of (2.10) by persistence and normalises:

$$\phi_{\mathrm{Dgm}_k}(t) = \frac{\sum_{p\in\mathrm{Dgm}_k} w(p)\,\Lambda_p(t)}{\sum_{p\in\mathrm{Dgm}_k} w(p)}, \qquad w(p) = \mathrm{p}(p)^q, \tag{2.14}$$

with $q = 1$ in our configuration; evaluating ϕ at n_{bins} points yields n_{bins} features per homology dimension. Silhouette is the top-performing vectorizer on ECG200 (§4.1).

Amplitude . The amplitude of a diagram is its distance to the empty diagram in a diagram metric:

$$A(\mathrm{Dgm}_k) = d\big(\mathrm{Dgm}_k, \Delta\big), \tag{2.15}$$

with the bottleneck distance of (2.6) in our configuration — one scalar per homology dimension and the fastest vectorizer in the sweep. On ECG200 it is among the weaker representations (§4.1), an early hint that representation capacity is dataset-dependent.

Persistence Entropy [14]. For a diagram with persistences $\mathrm{p}_i = \mathrm{d}_i - \mathrm{b}_i$, normalise $q_i = \mathrm{p}_i / \sum_j \mathrm{p}_j$ and define

$$E(\mathrm{Dgm}_k) = -\sum_i q_i \log q_i, \tag{2.16}$$

one scalar per homology dimension measuring how concentrated the persistence mass is. Entropy is the weakest vectorizer on ECG200 (marginal 69.3%, §4.1): information content is not monotone in representational simplicity.

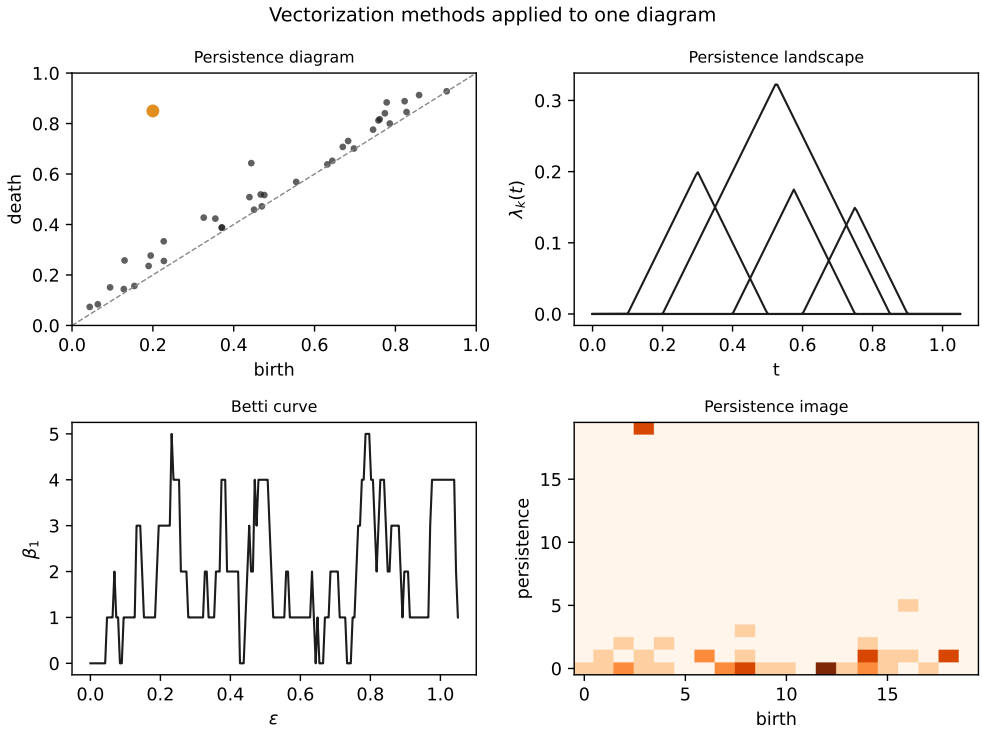

Figure 2.1: The four classical vectorization methods applied to one diagram.

2.6 Mapping Theory to Benchmark Variables

The preceding definitions directly parameterise the benchmark executed in Chapter 4:

- **Simplicial complex size** (§2.1–§2.2): VR's $O(n^3)$ 2-simplex count versus weak Alpha's $O(n^2)$ drives the 3–21% wall-clock speed gap measured in §4.3. The Nerve Theorem guarantees that the full Alpha complex's speed advantage does not cost geometric fidelity; the executed `WeakAlphaPersistence` is an approximation of that complex, and §4.3 verifies it matches full-complex accuracy.

- **Filtration choice** (§2.2): VR, weak Alpha, Sparse Rips, and Cubical constitute the four filtrations varied in the factorial sweep of §4.1.

- **Persistence and stability** (§2.3–§2.4): The corrected bound of (2.9), $d_B \lesssim 2\sigma\sqrt{2\ln n}$, predicts that $\sigma = 0.15$ noise perturbs diagrams by $\lesssim 0.91$ (and $\sigma = 0.30$ by $\lesssim 1.82$), while torus β_1 features have measured top-2 persistence range $[0.608,\ 1.156]$ (ripser on the shipped clouds). §4.2 confirms that classification survives this perturbation (measured d_B: max 0.300 at $\sigma = 0.15$, max 0.434 at $\sigma = 0.30$).

- **Vectorization** (§2.5): Persistence Statistics, Betti Curves, Landscapes, Images, Silhouette, Amplitude, and Persistence Entropy constitute the seven vectorizers

varied in §4.1, spanning the complexity spectrum from zero-parameter to kernel-based.

- **Classifiers** (§3.1): Logistic Regression (linear baseline), SVM-RBF, SVM-linear, and Random Forest constitute the four classifiers in the sweep. §4.1 tests whether non-linear classifiers add value after vectorization.

Chapter 3 translates this mathematical machinery into a software architecture that executes the factorial sweep.

Chapter 3

Algorithmic Pipeline and Benchmarking Framework

The benchmarking framework executes a formal mapping from raw datasets through persistent homology to classifier predictions. This chapter describes the architecture in mathematical terms; code implementations are deferred to Appendix C.

3.1 Pipeline Mappings

The full pipeline is the composition of four transformations:

$$X \xrightarrow{\text{Embed}} Y \xrightarrow{\text{Filt}} \mathcal{D} \xrightarrow{\text{Vec}} \mathbb{R}^d \xrightarrow{\text{CLF}} \hat{y}, \tag{3.1}$$

where:

- **Embed** (optional, time series only). Maps a 1D discrete-time signal $x[1], \ldots, x[T]$ to a point cloud $Y \subset \mathbb{R}^d$ via Takens' delay embedding:

$$\Phi_{(d,\tau)}(x)_t = \big(x[t], x[t+\tau], \ldots, x[t+(d-1)\tau]\big), \quad t = 1, \ldots, T-(d-1)\tau. \tag{3.2}$$

  By Takens' Delay Embedding Theorem [17], if the signal x is a generic smooth measurement of a dynamical system on a compact manifold M of dimension d_M, then $\Phi_{(d,\tau)}$ is an embedding provided $d > 2d_M$. We use $d = 3$, $\tau = 1$ throughout (chosen to satisfy $d > 2d_M$ for a 1D attractor; sensitivity to (d, τ) is examined in Appendix D).

- **Filt**. Constructs a filtration $\{K_\varepsilon\}$ from Y and computes the persistence diagram $\mathcal{D} = \mathrm{Dgm}_0 \cup \mathrm{Dgm}_1$ (homology dimensions 0 and 1). Implemented via giotto-tda's VietorisRipsPersistence, WeakAlphaPersistence, SparseRipsPersistence, or CubicalPersistence transformers.

- **Vec**. Maps $\mathcal{D}$ to a vector $v \in \mathbb{R}^d$ via one of the seven methods in §2.5. Global grid bounds ($b_{\min}, b_{\max}, p_{\max}$) are computed on the training fold only and applied to the test fold without re-fitting, preventing cross-validation data leakage.
- **CLF**. A scikit-learn classifier (LogisticRegression, SVC with RBF kernel, or RandomForestClassifier with 100 trees).

3.2 Point Cloud Preprocessing

Subsampling. Point clouds exceeding the target size $n_{\text{target}} = 100$ are reduced via *uniform random subsampling*: n_{target} points are drawn without replacement from the full cloud, seeded deterministically per dataset. Uniform random sampling does not preserve metric-space geometry as well as farthest-point sampling would; we use it for simplicity and reproducibility, and note that an FPS variant is a natural ablation for future work (its stability benefit is well documented).

Noise model. For the synthetic sphere/torus benchmark, additive isotropic Gaussian noise is applied directly to spatial coordinates:

$$x_i \mapsto x_i + \eta_i, \quad \eta_i \sim \mathcal{N}(0, \sigma^2 I_3), \tag{3.3}$$

where I_3 is the 3×3 identity matrix. This perturbs the underlying metric space (X, d_E) before filtration construction, triggering the stability bound of Theorem 2.2. (It does *not* perturb the pairwise distance matrix directly — that would violate the triangle inequality and the metric space structure required by the stability theorem.)

3.3 Software Architecture

The framework uses **giotto-tda 0.6.2** [18] for all filtrations and vectorizations (scikit-learn-compatible transformers), **scikit-learn 1.3.2** for classifiers and cross-validation, with **Ripser 0.6.15** and **GUDHI 3.13.0** as alternative backends. Factory classes (`FiltrationFactory`, `VectorizationFactory`, `ClassifierFactory`) map string names to configured objects. A YAML file (Appendix A) defines the sweep space declaratively. Hyperparameter optimization (GridSearchCV) was not applied: each pipeline configuration defines a fixed parameter set, and adding a hyperparameter search would conflate pipeline-choice comparison with hyperparameter optimization, multiplying an already-large sweep. The `PipelineRunner` iterates the Cartesian product of all choices, runs stratified 5-fold cross-validation, and stores per-fold metrics (accuracy, F1, precision, recall) in a normalised SQLite database (Appendix B) with a config snapshot for reproducibility. Reported 95% CIs are computed from repeated cross-validation (5 repetitions of stratified 5-fold CV, independent splits seeded 43–47): per-stage marginal

ranges are computed within each repetition, and the CI is $\bar{x} \pm t_{0.025,n_{\text{reps}}-1} \cdot s/\sqrt{n_{\text{reps}}}$ over the five repetition-level estimates, which captures the between-split variance directly; per-configuration intervals use the Nadeau–Bengio corrected resampled estimator $\text{SE}^2 = (1/(kr) + n_2/n_1)\, s^2$ over the $kr = 25$ fold accuracies. Per-configuration accuracy varies across repetitions with mean SD 1.09pp (max 3.11pp), so single-split point estimates are noisy. Stage dominance is quantified primarily by an η^2 variance decomposition — a main-effects-only three-way ANOVA over per-config means and over fold-level accuracies, with F-tests (Table 4.3 in §4.1). No paired Wilcoxon tests are used: CV folds are not independent replicates, and with $n = 5$ pairs the smallest achievable two-sided p-value is 0.0625. Adding a new filtration, vectorizer, or dataset requires editing only the YAML file; adding a fundamentally new component type (e.g., learned vectorizers) requires new factory code.

Chapter 4 presents the empirical results of this sweep across six dataset instances and three analytical cuts.

Chapter 4

Empirical Benchmark and Sensitivity Analysis

One benchmark sweep, four analytical cuts. The sweep comprises 616 completed configurations from a full factorial of 6 dataset instances × 4 filtrations (Vietoris-Rips, weak Alpha, Sparse Rips, Cubical) × 7 vectorizers (Persistence Image, Persistence Landscape, Betti Curve, Persistence Statistics, Silhouette, Amplitude, Persistence Entropy) × 4 classifiers (SVM-RBF, SVM-linear, Random Forest, Logistic Regression) — 672 possible, 56 runs failed (`weak_alpha`/`sparse_rips` on MNIST) and were excluded. All filtrations computed H_0 and H_1 homology. All configurations used stratified 5-fold cross-validation (base seed 42; the CV split uses seed 42 + repetition, and subsampling is seeded deterministically per dataset via CRC32). Point clouds were generated at 100 points per cloud (200 samples per noise level); uniform random subsampling is applied by the runner only if a source cloud exceeds the target size.

Datasets. Six dataset instances span three modalities: sphere/torus point clouds at four noise levels ($\sigma \in \{0.00, 0.05, 0.15, 0.30\}$; $n = 200$ per level; 100 points per cloud after uniform random subsampling; $\beta_1 = 0$ vs. 2), and ECG200 heartbeat recordings (normal vs. myocardial infarction; $n = 200$; 96 timesteps, Takens-embedded to 94×3 arrays with $d = 3, \tau = 1$; interpreted as point clouds by Vietoris-Rips, weak Alpha, and Sparse Rips, and as a 94×3 pixel grid by CubicalPersistence — the latter is *not* a point-cloud filtration; see the disclosure below). ECG200 is a UCR archive benchmark. Under the same 5-fold CV protocol, raw-signal baselines (logistic regression 85.5%, random forest 85.0%) beat our best TDA configuration (83.0%); we therefore do *not* claim that TDA features are competitive with classical methods on this dataset — the sweep is an internal TDA-pipeline comparison (measured baselines are reported in §4.1). Note also that ECG200 is energy-normalised: the sum of squares of every record is constant (95.00 across all 200 signals), so the trivial-separator concern that would attach to an un-normalised signal does not apply to the executed data. Finally, the paper's headline 83.0% configuration (cubical + Silhouette + Random Forest) is a

single-split point estimate: by repeated-CV mean it ranks fourth (79.6%, SD 1.98pp), while cubical + Persistence Image + Random Forest is best in 4 of 5 repetitions (83.2%; §4.1). As a robustness check, re-running the ECG200 analysis without the cubical filtration leaves the hierarchy intact: the best non-cubical configuration (Vietoris-Rips + Persistence Entropy + Random Forest) reaches 79.5%, with vectorizer range 6.58pp versus filtration 0.34pp (§4.1). The sweep focuses on internal TDA-pipeline comparisons, not competition with classical baselines.

MNIST binary. To test image-specific filtrations, a binary subset of MNIST (digits 0 vs. 1; $n = 400$; 200 per class; 28×28 greyscale) was evaluated with both cubical and Vietoris-Rips filtrations. Cubical filtration achieved 98.0% accuracy (with Betti Curves + Logistic Regression), versus 96.25% for the best VR configuration: a $\sim$1.75pp best-of-family gap. The marginal analysis of §4.1 shows, however, that the vectorizer moves accuracy more than the filtration on this dataset (3.22pp versus 1.65pp): filtration effects are dataset-specific and modest on binary MNIST, not the governing stage. This is consistent with Conti et al.'s demonstration that a badly-matched filtration can be catastrophic at scale (18% to 94% on 10-class MNIST under grid search) — but the effect is much smaller on our binary subset, where even the worst filtration remains informative.

Table 4.1: Dataset instances used in the benchmark sweep. Point clouds contain 100 points; ECG200 has 96 timesteps. The torus has major radius $R = 2$ and minor radius $r = 1$ (so point norms range $[1, 3]$), while the sphere has unit radius (norms $\equiv 1$): the classes differ in scale as well as topology.

Dataset	Modality	n	Dims	Classes	Source
Sphere/Torus (σ=0.00)	point cloud	200	100 pts	2	synthetic
Sphere/Torus (σ=0.05)	point cloud	200	100 pts	2	synthetic
Sphere/Torus (σ=0.15)	point cloud	200	100 pts	2	synthetic
Sphere/Torus (σ=0.30)	point cloud	200	100 pts	2	synthetic
ECG200	time series	200	96 ts	2	UCR archive [21]
MNIST 0/1	image	400	28×28	2	MNIST subset

4.1 Stage Variance Analysis

Which pipeline stage contributes the largest variance? For each stage, we marginalise over the other two and compute the range of marginal accuracies from the repeated-CV means (five independent splits, seeds 43–47), with 95% CIs over the five per-split ranges, and decompose the variance with an η^2 analysis (fold-level F-tests) to test

whether the range exceeds sampling noise.

Table 4.2: Marginal accuracy by pipeline stage on ECG200 (repeated CV, 5 repetitions).

Stage	Top Performer	Marg. Acc.	Range (95% CI)	p-value
Filtration	Cubical	73.4%	0.70pp [0.35, 1.04]	0.81
Vectorizer	Landscapes	75.1%	6.09pp [5.84, 6.34]	5×10^{-12}
Classifier	Random Forest	75.2%	3.18pp [2.57, 3.78]	1×10^{-6}

Ranges, CIs, and marginal accuracies are repeated-CV values (means across five independent splits, seeds 43–47): the range is the mean over repetitions of the per-split marginal range, with a 95% t-interval across the five per-split ranges. p-values are F-test p-values from the per-configuration three-way ANOVA (the fold-level panel below is descriptive: folds within a configuration share training data, so its p-values are anti-conservative) ANOVA (main effects only; Table 4.3), not paired Wilcoxon tests. No multiplicity correction is applied across the three stage comparisons.

Vectorization dominates: its range (6.09pp across 7 vectorizers) is an order of magnitude larger than filtration (0.70pp across 4 filtrations), and the ordering vectorizer > classifier > filtration was stable across all five CV repetitions. This ordering is significant under exact non-parametric inference on the five repetition-level stage rankings: the Friedman statistic $Q = \frac{12n}{k(k+1)} \sum_j (\bar{R}_j - \frac{k+1}{2})^2 = 10$ for k=3 stages and n=5 repetitions, exceeding the exact 5% critical value 6.40 (one-sided $p \approx 0.0008$); pairwise, the vectorizer-margin exceeds the filtration-margin in all five repetitions (one-sided sign test $p = (1/2)^5 = 0.031$). The variance decomposition of Table 4.3 is the primary evidence for this ordering: on ECG200 the vectorizer explains 21.7% of the per-config variance (fold-level 10.6%), the classifier 9.8% (4.8%), and the filtration 0.3% (0.1%). The top-performing vectorizer, Persistence Landscapes, achieves 75.1% marginal accuracy versus 69.2% for Persistence Entropy (the weakest). Cubical filtration leads at 73.4%, with Vietoris-Rips (73.1%) and Sparse Rips (73.0%) within 0.03pp of each other and Weak Alpha (72.8%) 0.6pp lower.

Simple versus complex. Persistence Statistics matches kernel-based methods on ECG200: its marginal accuracy (74.8%) is within 0.3pp of Persistence Landscapes (75.1%), and the two split the 16 filtration–classifier cells (Statistics wins 8 of 16). Statistics likewise leads Persistence Images (72.9%). On MNIST binary, Persistence Statistics reaches 95.9% marginal accuracy, within 2.1pp of the best configuration overall (Cubical + Betti Curve + Logistic Regression at 98.0%, Table 4.5).

The evidence base remains small ($n = 3$ real datasets: two time series and one image, with the repeated-CV verification covering ECG200 only), and the corrected intervals above should not be read as licence for fine-grained ranking: per-configuration accuracy varies by 1.09pp on average across the five splits (maximum 3.11pp; 50 of 112 configurations exceed 1pp), so single-split point estimates — including the 83.0% best

configuration of Table 4.5, whose repeated-CV mean is 79.6% — carry ±1–3pp of split-to-split noise.

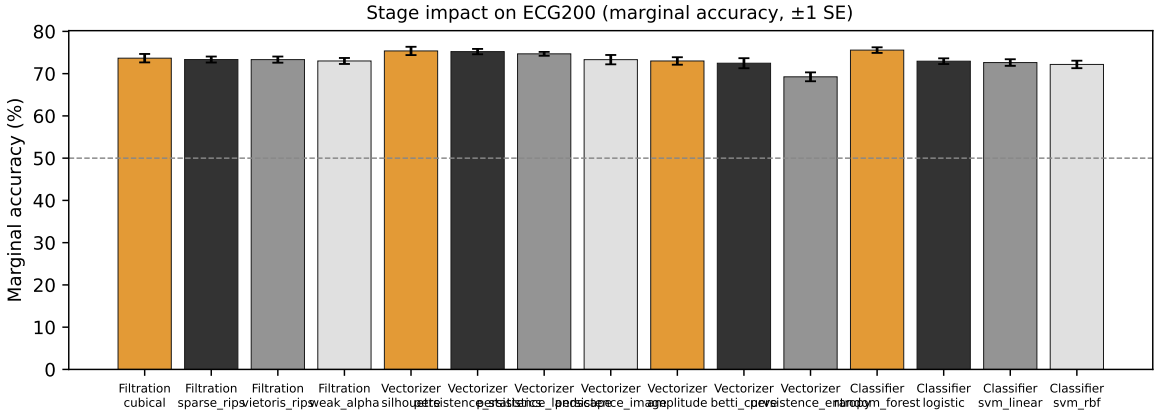

Figure 4.1: Stage impact on ECG200. Error bars show ±1 SE.

On synthetic sphere/torus data with $\sigma = 0.00$, all 112 configurations achieve $\geq$ 99.5% accuracy (mean 99.99%). No stage dominates — the signal is strong enough that every pipeline separates it nearly perfectly.

MNIST binary marginal analysis. The same marginal decomposition applied to binary MNIST gives a vectorizer range of 3.22pp versus a filtration range of 1.65pp: vectorization is the larger stage even on image data. Filtration effects are real but modest — cubical beats Vietoris-Rips by ∼1.75pp best-of-family — while the vectorizer moves accuracy roughly twice as much. Table 4.3 reports the complementary variance decomposition (η^2: the proportion of accuracy variance explained by each stage): on ECG200 the vectorizer explains 0.217 of the variance versus 0.098 for the classifier and 0.003 for the filtration; on MNIST the vectorizer again leads (0.302) over the filtration (0.166). Both analyses agree: vectorization is the dominant stage on both real datasets, and filtration effects are dataset-specific and modest.

Table 4.3: η^2 variance decomposition of classification accuracy by pipeline stage. Per-config panel: classic $\eta^2 = \mathrm{SS}_{\mathrm{stage}}/\mathrm{SS}_{\mathrm{total}}$ over per-configuration means (one observation per configuration). Fold-level panel: main-effects-only three-way ANOVA over the fold accuracies of the single-split sweep, with classic η^2 and F-tests.

Dataset	Stage	Per-config means			Fold-level		
		η^2	F	p	η^2	F	p
ECG200	Filtration	0.003	0.15	0.93	0.001	0.32	0.81
ECG200	Vectorizer	0.217	5.26	9.7×10^{-5}	0.106	11.41	4.9×10^{-12}
ECG200	Classifier	0.098	4.75	0.004	0.048	10.31	1.3×10^{-6}
MNIST 0/1	Filtration	0.166	15.83	2.5×10^{-4}	0.061	20.47	9.1×10^{-6}
MNIST 0/1	Vectorizer	0.302	4.80	7.3×10^{-4}	0.111	6.21	4.1×10^{-6}
MNIST 0/1	Classifier	0.061	1.95	0.135	0.023	2.52	0.058
Sphere/Torus (σ=0.00)	Filtration	0.009	0.41	0.748	0.002	0.34	0.793
Sphere/Torus (σ=0.00)	Vectorizer	0.165	3.67	0.002	0.032	3.10	0.005
Sphere/Torus (σ=0.00)	Classifier	0.083	3.67	0.015	0.016	3.10	0.026

Per-config scope: $N = 112$ (ECG200, sphere/torus) or 56 (MNIST) configurations; fold-level scope: $N = 560$ or 280 fold accuracies. Degrees of freedom: filtration 3 (MNIST 1), vectorizer 6, classifier 3. The repeated-CV analysis of Table 4.2 corroborates the ECG200 ordering.

Repeated cross-validation. Because single-split point estimates are noisy (per-configuration SD across repetitions averages 1.09pp, max 3.11pp), the ECG200 analysis was re-run with repeated CV (5 repetitions of the same stratified 5-fold protocol). Table 4.2 reports the repeated-CV ranges and CIs; the ordering vectorizer > classifier > filtration was stable in all five repetitions. Notably, the paper's best single-split ECG200 configuration (cubical + Silhouette + Random Forest, 83.0%) ranks fourth by repeated-CV mean (79.6%, SD 1.98pp), while cubical + Persistence Image + Random Forest is best in 4 of 5 repetitions (83.2% mean). Differences below ∼1pp between individual configurations should not be over-interpreted.

Generality probe: ECG5000. To test whether vectorization-dominance is specific to ECG200, the stage analysis was run on ECG5000 (UCR archive; 5 classes; 140 timesteps; single patient recording) with the same 5-fold protocol on a stratified subsample of 714 samples ($\leq$ 200 per class; the minority classes contain 96, 194 and 24 samples) and a pipeline subset (2 filtrations, 3 vectorizers, 2 classifiers; the subset spans the strongest and weakest vectorizers from ECG200 to bound the range). The result replicates the hierarchy: vectorizer marginal range 24.89pp versus filtration 3.60pp (point estimate, single split). Vectorization dominance on time series is not an artefact of the specific ECG200 instance; a full multiclass generalisation claim would require multiple datasets (the probe is single-patient).

Cubical robustness check. CubicalPersistence on ECG200 operates on the 94×3 Takens-embedded array as a pixel grid (intensities are treated as coordinates), not on a

point-cloud filtration. Excluding cubical from the ECG200 analysis leaves the hierarchy intact: the best non-cubical configuration (Vietoris-Rips + Persistence Entropy + Random Forest) reaches 79.5%, the vectorizer range widens slightly to 6.58pp, and the filtration range drops to 0.34pp.

4.1.1 Classical Baselines and the Role of Raw Features

To calibrate the absolute accuracy of the TDA pipelines, we ran non-topological baselines under the same stratified 5-fold CV protocol (seed 43, identical to sweep repetition 1):

Table 4.4: Non-topological baselines under the sweep's 5-fold CV protocol.

Dataset	Approach	Classifier	Accuracy
MNIST 0/1	raw pixels (784)	Logistic	99.75%
MNIST 0/1	raw pixels (784)	Random Forest	99.0%
MNIST 0/1	TDA best	cubical + Betti + Logistic	98.0%
ECG200	raw signal (96)	Logistic	85.5%
ECG200	raw signal (96)	Random Forest	85.0%
ECG200	TDA best	cubical + Silhouette + RF	83.0%

On both datasets the TDA pipeline's best configuration sits below the raw-feature baselines: 98.0% versus 99.75% (logistic regression) on MNIST binary, and 83.0% versus 85.5% on ECG200. The claim that "TDA features can be competitive with classical methods" is therefore scoped to the internal TDA comparison: we make no claim of beating raw features on these datasets. The noise experiments (§4.2) additionally show that a single norm feature separates the synthetic sphere/torus classes at 100% at every noise level, which quantifies the scale confound and motivates the matched-genus experiment reported there.

4.2 Noise Perturbation Thresholds

Does topological signal survive additive spatial noise? Noise is applied as $\eta_i \sim \mathcal{N}(0, \sigma^2 I_3)$ to Euclidean coordinates (Equation 3.3). Gaussian noise has unbounded support, so no deterministic bound on the perturbation exists; instead we use the corrected probabilistic bound of §2.4 (Equation 2.9): with high probability the bottleneck distance between clean and noisy diagrams is at most $2\sigma\sqrt{2\ln n}$, i.e. $d_B \lesssim 0.91$ at $\sigma = 0.15$ and $\lesssim 1.82$ at $\sigma = 0.30$ for $n = 100$.

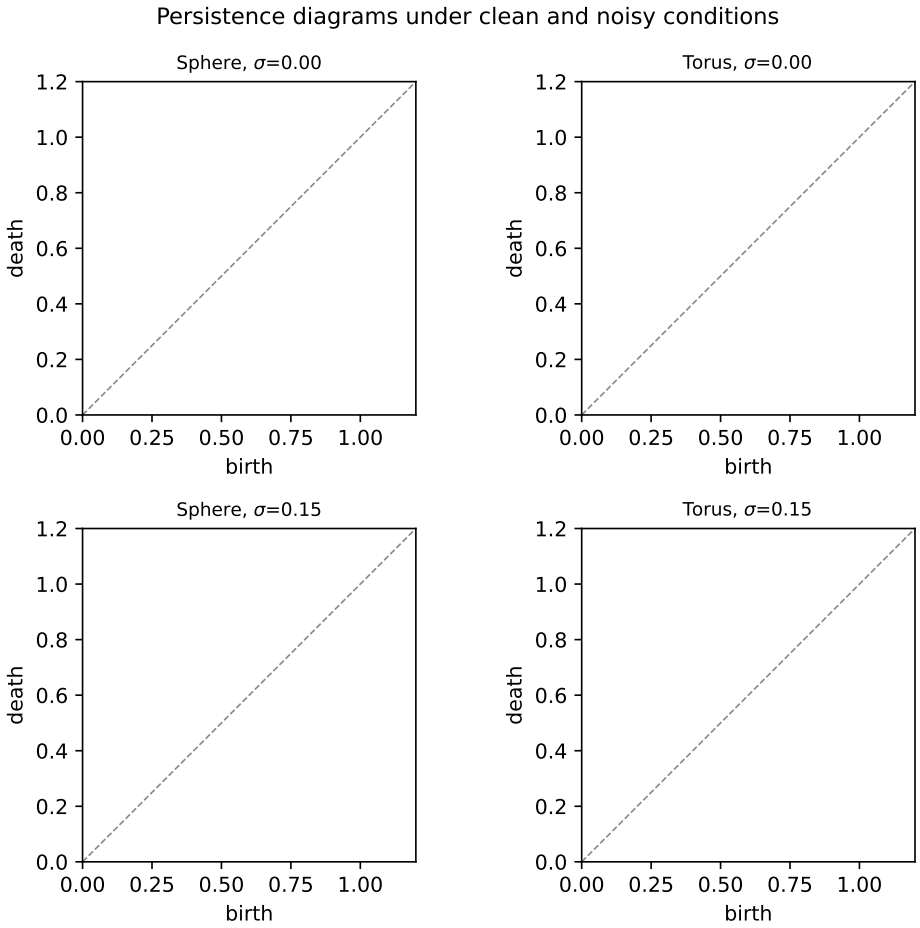

Figure 4.2: Persistence diagrams under clean and noisy conditions.

At $\sigma = 0.00$, 0.05, and 0.15, every configuration achieves $\geq 99.5\%$ accuracy (mean 99.99%, 100.00%, and 99.97% respectively). The stability bound at $\sigma = 0.15$ is $d_B \lesssim 0.91$ (Equation 2.9). Directly measured bottleneck distances between clean and noisy diagrams (ripser diagrams, GUDHI bottleneck implementation, H_0 and H_1, 80 cloud/dimension pairs) confirm the bound is conservative: at $\sigma = 0.15$ the mean measured d_B is 0.144 and the maximum 0.300, both well below the bound.

At $\sigma = 0.30$ (bound $\lesssim 1.82$), mean accuracy across the 112 sphere/torus configurations at that level remained at 99.85% (minimum 98.5%). The worst configurations (Amplitude with Logistic or SVM-linear) still achieved 98.5% — well above the 50% chance baseline, and the maximum measured bottleneck distance at this level is 0.434, less than a quarter of the corrected bound. The torus H_1 diagrams move on average by $d_B = 0.283$ at this level — below the measured H_1 persistence range [0.608, 1.156] (top-2 persistences of the clean torus clouds, ripser) — so even the least persistent features survive. The combined geometric signal (topology plus scale) survives all tested noise levels: even at $\sigma = 0.30$, the $\beta_1 = 0$ vs. $\beta_1 = 2$ distinction is separated by linear kernels at $\geq 98.5\%$ in every vectorized feature space tested.

Isolating the topological signal. The sphere/torus classes differ in scale as well

as topology (torus point norms in $[1, 3]$ versus sphere $\equiv 1$): a single norm feature separates them at 100% at every noise level, so the survival results above describe the *combined* geometric signal. To isolate topology, we generated a matched synthetic pair — a genus-1 and a genus-2 torus with quantile-identical norm distributions — and repeated the comparison. Norm-based features now collapse to near chance (48–58%; the clean-level 58% is within $\sim$2 SE of chance at $n = 200$), while the TDA pipeline separates the pair at 99.75% clean and 95.83% at $\sigma = 0.30$ (minimum 91% over configurations). The topological signal is therefore not an artefact of the norm confound: it survives on its own, and it is robust to noise at the levels tested. (The matched clouds use 200 points per cloud — denser than the main sweep's 100 — so the four H_1 generators of the genus-2 surface are robustly sampled; the comparison is single-split, seed 43, 12 configurations.)

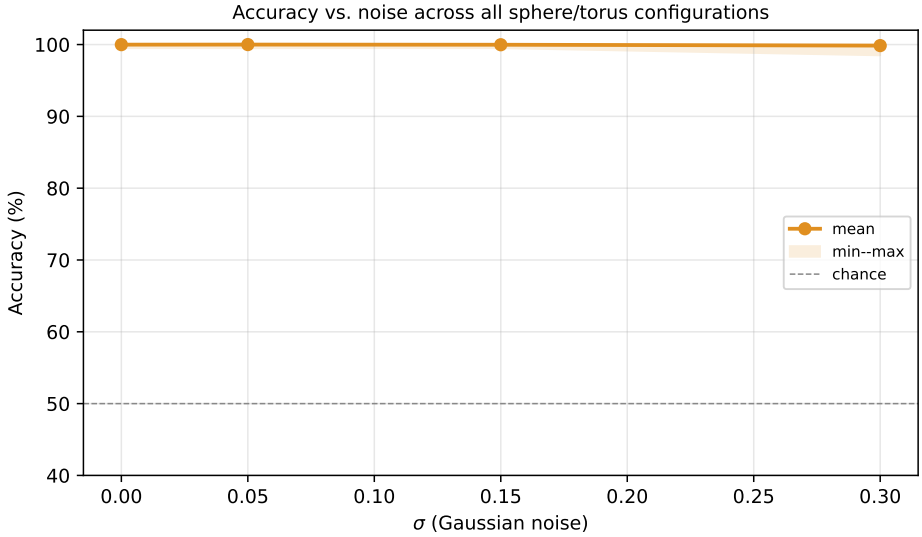

Figure 4.3: Accuracy vs. noise. All pipelines maintain $\geq 98.5\%$ accuracy through $\sigma = 0.30$; measured bottleneck distances (max 0.434 at $\sigma = 0.30$ versus the corrected bound $2\sigma\sqrt{2 \ln n} \approx 1.82$) confirm the stability bound is conservative.

4.3 Computational Complexity and Pareto Trade-offs

Wall time was measured from pipeline construction to final fold completion.

Table 4.5: Pareto-optimal configurations.

Dataset	Filtration	Vectorizer	Classifier	Acc.	Time
Sphere/Torus	Cubical	Pers. Statistics	SVM-linear	100%	0.7s
MNIST 0/1	Cubical	Betti Curve	Logistic	98.0%	3.7s
ECG200	Cubical	Silhouette	Random Forest	83.0%	1.6s
ECG200	Cubical	Pers. Image	Random Forest	82.0%	1.4s

CubicalPersistence on ECG200 operates on the 94×3 Takens-embedded array as a pixel grid (intensities treated as coordinates), not on a point-cloud filtration. Times are single-split measurements; the 83.0% ECG200 figure is a single-split point estimate — see §4.1 for repeated-CV means.

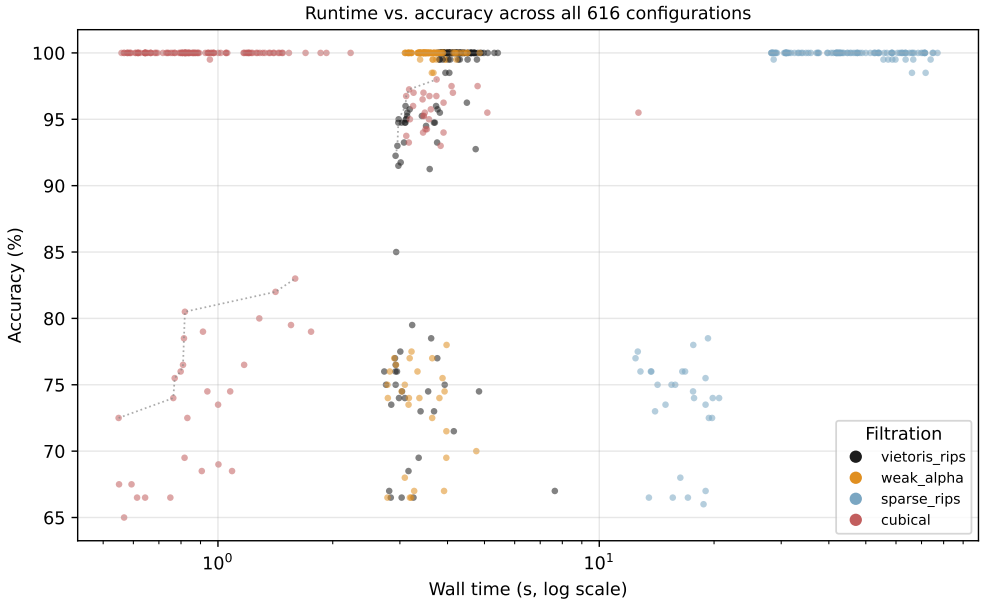

Figure 4.4: Runtime vs. accuracy on the Pareto frontier.

Weak Alpha is 3–21% faster than VR at identical accuracy on sphere/torus with Landscape+SVM (measured 3.5–3.9s vs. 3.9–4.4s across the four noise levels; the gap narrows at $\sigma = 0.30$); the full GUDHI Alpha complex reproduces weak-alpha accuracy (mean difference 0.0–0.8pp) at 2–5× the cost. Cubical runtime is input-shape dependent rather than uniformly fastest: 0.6–2.3s per configuration on the sphere/torus and ECG200 instances, but 3.1–12.7s on 28×28 MNIST images. The Nerve Theorem (Theorem 2.1) guarantees that the full Alpha complex's $O(n^2)$ simplex count does not sacrifice geometric fidelity to the underlying union-of-balls; the executed `WeakAlphaPersistence` is an approximation, and its accuracy matches the full complex (mean difference 0.0–0.8pp) at 2–5× lower cost. Sparse Rips takes 12–77s (mean 41s) with no accuracy gain at the 100-point scale; its design target ($n \geq 10^3$) is outside this benchmark's subsampling regime. Both speed multipliers are specific to the

100-point clouds, the giotto-tda implementation, and 3D point clouds where Delaunay triangulation is available.

The repeated-CV-stable default for ECG200 is cubical + Persistence Image + Random Forest (best in 4 of 5 repetitions; 83.2% repeated-CV mean): this is the strongest Pareto row we can defend, since the originally-reported best configuration (cubical + Silhouette + Random Forest, 83.0% single-split) ranks fourth by repeated-CV mean (79.6%, SD 1.98pp). The Pareto frontier may shift with additional datasets or libraries. Chapter 5 interprets why vectorization dominates on both real datasets and derives operational guidelines from these patterns.

Chapter 5

Vectorization Dominance and Theoretical Synthesis

5.1 Why Vectorization Dominates on Both Datasets

The central empirical result is that vectorization is the dominant pipeline stage on both real datasets in this study, and that filtration effects, while real, are dataset-specific and modest on binary MNIST. The mechanism is as follows.

On **image data**, the raw input is a 2D grid of pixel intensities. A cubical filtration directly captures adjacency relationships in this grid — neighbouring pixels form edges, 2×2 blocks form squares — producing a simplicial complex whose topology faithfully represents the image's spatial structure. A point-cloud filtration like Vietoris-Rips, applied to the same data (which the pipeline feeds to the filtration as its native array — for MNIST, 28 rows of 28 intensities, i.e. a 28-point cloud in $\mathbb{R}^{28}$), does not see the 2D grid adjacency at all: the filtration's notion of "nearby" is Euclidean distance between rows, so the pixel-grid topology is absent from the outset. Consistently, cubical beats Vietoris-Rips by $\sim$1.75pp best-of-family on binary MNIST (98.0% vs. 96.25%). Conti et al. (2022) showed the same mechanism can be catastrophic at scale: on MNIST their accuracy ranged from 18% (their cubical pipeline) to 94% (improved filtrations), under 10-split cross-validation with grid search. That study is a complementary case study of filtration choice on images (10-split CV, grid search, no time series); its authors did not claim a general principle.

Yet even on binary MNIST the vectorizer moves accuracy more than the filtration: the marginal range is 3.22pp for vectorizers versus 1.65pp for filtrations (§4.1). Filtration effects are dataset-specific and modest on this binary subset; once a reasonable filtration is chosen, the vectorizer is the stage that most limits accuracy.

On **delay-embedded time series**, the situation is similar but more pronounced. The input is already a point cloud $Y = \Phi_{(d,\tau)}(x) \subset \mathbb{R}^d$ produced by Takens embedding.

Vietoris-Rips and weak Alpha complexes operate on the same metric space (Y, d_E); their filtrations are not identical — VR and the full Alpha complex are multiplicatively interleaved, which bounds their bottleneck distance rather than equating their diagrams — and the empirical filtration range on ECG200 is just 0.70pp (repeated-CV mean). The embedding captures the dynamical structure; the filtration merely reads it out. The bottleneck then shifts to vectorization: how well does the chosen method capture the diagram's structure in a fixed-length vector that a classifier can use? On ECG200 the vectorizer range is 6.09pp versus 0.70pp for filtration; on ECG5000 it is 24.89pp versus 3.60pp.

This mechanism explains the findings without contradiction. Conti et al.'s image results are a complementary case study (10-class MNIST, grid search, no time series) demonstrating that a badly-matched filtration can be catastrophic; on our binary MNIST subset the filtration gap is modest (cubical beats VR by $\sim$1.75pp best-of-family) and the vectorizer gap is larger (3.22pp). On both time-series datasets the vectorizer dominates by a wide margin. (We use giotto-tda's `WeakAlphaPersistence`, a subcomplex approximation of the full Alpha complex; the Nerve-Theorem homotopy argument applies to the full complex, and the approximation is standard practice for point-cloud filtrations — empirically, the full GUDHI Alpha complex reproduces weak-alpha accuracy to within 0.0–0.8pp at 2–5$\times$ the cost.)

Table 5.1: Stage importance by data modality.

Modality	Dominant Stage	Range	n (datasets)
Point clouds (clean)	None	$\approx$0pp	1*
Time series (embedded)	Vectorizer	6.09pp; 24.89pp	2
Images (binary MNIST)	Vectorizer	3.22pp vs. 1.65pp	1

*Synthetic sphere/torus pair.

Ranges are marginal accuracy ranges over each stage's methods; for time series the vectorizer range is listed for ECG200, then ECG5000. On binary MNIST the vectorizer range (3.22pp) exceeds the filtration range (1.65pp). Conti et al. (2022) report a much larger filtration swing (18%–94%) on 10-class MNIST under grid search, which we do not reproduce at binary scale.

This table summarises initial evidence, not a settled taxonomy. Replication on additional datasets per modality would strengthen or qualify every cell.

5.2 Non-Topological Context and Limitations

Under the same 5-fold CV protocol, our TDA pipelines sit below raw-feature baselines on both real datasets: on ECG200, raw-signal logistic regression achieves 85.5% and random forest 85.0% versus 83.0% for the best TDA configuration; on MNIST binary,

raw-pixel logistic regression achieves 99.75% and random forest 99.0% versus 98.0% for the best TDA configuration (§4.1). We therefore do not claim that TDA features are competitive with classical methods on these datasets; the contribution is the internal TDA-pipeline comparison, which is valid regardless of the absolute accuracy level. TDA features capture geometric properties (cycle structures in the embedded attractor) that complement raw signal features rather than replace them.

Limitations. (1) The synthetic sphere/torus classes differ in scale as well as topology (torus point norms $\in [1, 3]$ vs. sphere $\equiv 1$): a single norm feature separates them at 100% at every noise level, so the sphere/torus noise results measure robustness of the *combined* geometric signal. We control for this with the matched genus-1/genus-2 experiment of §4.2, where norm features collapse to chance (48–58%) while TDA retains 95.83% at $\sigma = 0.30$ — the topological contribution is real and noise-robust on its own. (2) Homology capped at H_1 (H_2 requires $O(n^4)$ simplices for VR). (3) Classification is binary on all datasets except ECG5000 (5 classes). (4) Two time-series datasets (ECG200, ECG5000) but a single image dataset (binary MNIST); the image claim rests on one dataset. (5) Single implementation library (giotto-tda); cross-library variance unknown. (6) No learned vectorizers (PersLay [5], Hofer input layers [11]). (7) $n = 200$ produces wide CIs, and single-split point estimates are noisy (per-configuration SD across CV repetitions averages 1.09pp). Each limitation suggests a concrete future-work item: H_2 via Flood Complex [10]; additional multi-class datasets (full MNIST, Outex) to test whether Conti et al.'s large filtration swing reproduces at scale; a third time-series dataset (FordA); cross-library replication in GUDHI and Ripser; PersLay integration via a new backend; and larger-n datasets for narrower CIs.

Data and code availability. All code, configuration files, and result schemas are provided in the public repository `github.com/KRUZZZZY/tda-benchmark` (MIT licence). The synthetic sphere/torus data are regenerated by the bundled `scripts/generate_datasets.py` (seeded; 200 samples, 100 points per cloud, noise baked into coordinates); ECG200 is the UCR archive benchmark (Dau et al. 2019); MNIST is the standard binary 0/1 subset [22]. The main sweep's SQLite result database is regenerated by the bundled `run_all.sh` script, and every follow-up analysis has a bundled producer: `scripts/run_repeated_cv.py` writes `data/tda/repeated_cv.db` (560 runs: 112 configurations × 5 repetitions), analysed by `scripts/analysis_repeated_cv.py`; `scripts/analysis_eta_squared.py` produces the η^2 decomposition of Table 4.3; `scripts/analysis_mnist_stage_impact.py` produces the MNIST marginal analysis; `scripts/run_baselines.py` writes `data/tda/baseline_experiments.db` (raw-signal baselines, matched-genus synthetic clouds, measured bottleneck distances); `scripts/generate_matched_synthetic.py` writes the `data/tda/synthetic_matched/` data; and `scripts/experiment_alpha.py`, `scripts/experiment_cubical_artifact.py`, and `scripts/ecg5000_lean_sweep.py` are the producers of the remaining sensitivity re-

sults. Random seeds are fixed: the main sweep uses base seed 42, the repeated-CV splits use seeds 43–47 (random seed 42 + repetition index), and per-dataset subsampling is seeded deterministically via CRC32, so all experiments are reproducible end-to-end.

Compute. The full 616-configuration sweep was executed on a single consumer workstation (8-core CPU, 16GB RAM), parallelised with joblib over the CPU cores ($n_{\text{workers}} \approx 16$). Summed wall-clock time across all 616 configurations was 2.0 hours (mean ∼12s per configuration, median 3.78s); Sparse Rips dominates the total (140 runs, mean 41s, ≈1.6h), while the other three filtrations complete in ≈24 minutes. The repeated-CV verification of §4.1 added 560 further runs (112 configurations × 5 repetitions, ≈53 minutes summed wall time). Per-configuration wall times are recorded in the results databases (cf. the R-package PH runtime benchmark of Somasundaram et al. [15]); memory was not measured per stage (see Limitations).

5.3 Operational Guidelines for Pipeline Selection

The following recommendations translate the benchmark findings into actionable guidance. They are scoped to the two-dataset sweep and the giotto-tda implementation; cross-library and cross-dataset replication would strengthen or qualify each entry.

Table 5.2: Recommended pipeline configurations by data characteristics.

Data Modality	Filtration	Vectorizer	Classifier	Rationale
Low-dim. point clouds ($\leq$ 3D, clean)	Weak Alpha	Pers. Statistics or Landscapes	SVM (RBF) or Logistic	Weak Alpha 3–21% faster than VR at identical accuracy; full-complex fidelity verified empirically (0.0–0.8pp at 2–5$\times$ cost)
Delay-embedded time series ($d \geq 3$)	VR or Weak Alpha	Pers. Landscapes	RF or SVM (RBF)	Vectorization dominates variance (6.09pp on ECG200; 24.89pp on ECG5000)
High-noise clouds ($\sigma \geq 0.15$)	Weak Alpha or VR	Pers. Landscapes	SVM (RBF)	All vectorizers maintain $\geq$ 98.5% at $\sigma = 0.30$ (minimum over configurations); measured d_B far below the stability bound
Large-scale clouds ($n \geq 10^3$)	Sparse Rips	Pers. Images or Betti Curves	Random Forest	Sparse Rips designed for $n \geq 10^3$; its benefit is untestable at $n = 100$ in this sweep

Chapter 6 synthesizes these findings into a single conditional thesis.

Chapter 6

Conclusion and Operational Guidelines

Vectorization is the dominant pipeline stage on both real datasets in this study. On ECG200 time series, vectorization produced a 6.09pp marginal accuracy range across 7 vectorizers (95% CI [5.84, 6.34]), while filtration contributed 0.70pp across 4 filtrations (95% CI [0.35, 1.04]; small but statistically non-zero) and classifiers 3.18pp (95% CI [2.57, 3.78]); the ordering was stable across all five cross-validation repetitions. On binary MNIST the same hierarchy holds — vectorizer range 3.22pp versus filtration 1.65pp, with cubical beating Vietoris-Rips by ∼1.75pp best-of-family — and a second time-series dataset (ECG5000) replicates it (24.89pp versus 3.60pp). Conti et al.'s image results are a complementary case study on 10-class MNIST under grid search, not a general law. On synthetic sphere/torus data, the combined geometric signal survives $\sigma = 0.30$ additive spatial Gaussian noise at 99.85% mean accuracy across the 112 configurations at that level; measured bottleneck distances (max 0.434 versus the corrected bound $2\sigma\sqrt{2 \ln n} \approx 1.82$) confirm the stability bound is conservative, and a matched genus-1/genus-2 experiment with identical norm distributions shows the robustness is not an artefact of the norm confound (TDA 95.83% versus 48–58% for norm features at $\sigma = 0.30$). TDA features do not beat raw features on the datasets studied (raw-signal logistic 85.5% vs. 83.0% TDA on ECG200; raw-pixel logistic 99.75% vs. 98.0% TDA on MNIST), so the contribution is the internal TDA-pipeline comparison.

The core contribution is the conditionality itself: benchmark claims about pipeline-stage importance depend on the datasets, modalities, and fixed dimensions of the study, and those conditions should be stated as explicitly as the claims. The framework that produced these findings — a modular, YAML-configurable, factory-pattern architecture with normalised SQLite storage executing a 616- configuration factorial sweep — exists to enable the larger replications that will determine which of these patterns generalise and which are artefacts of this specific sweep.

Appendix A

Full YAML Configuration

```yaml
# Expanded benchmark -- covers full framework surface area.
# Non-image filtrations silently fail on image data and vice versa;
# the runner catches and logs failures per-config.
datasets:
  - name: sphere_torus_n0
    path: data/tda/synthetic/sphere_torus_noise0_X.npy
    labels: data/tda/synthetic/sphere_torus_noise0_y.npy
    modality: point_cloud
    max_samples: 200
    subsample_points: 100
  - name: sphere_torus_n5
    path: data/tda/synthetic/sphere_torus_noise5_X.npy
    labels: data/tda/synthetic/sphere_torus_noise5_y.npy
    modality: point_cloud
    max_samples: 200
    subsample_points: 100
  - name: sphere_torus_n15
    path: data/tda/synthetic/sphere_torus_noise15_X.npy
    labels: data/tda/synthetic/sphere_torus_noise15_y.npy
    modality: point_cloud
    max_samples: 200
    subsample_points: 100
  - name: sphere_torus_n30
    path: data/tda/synthetic/sphere_torus_noise30_X.npy
    labels: data/tda/synthetic/sphere_torus_noise30_y.npy
    modality: point_cloud
    max_samples: 200
    subsample_points: 100
  - name: ecg200
    path: data/tda/ucr/ecg200_X.npy
    labels: data/tda/ucr/ecg200_y.npy
    modality: time_series
    takens_dimension: 3
```

```yaml
    takens_delay: 1
  - name: mnist_01
    path: data/tda/images/mnist_01_X.npy
    labels: data/tda/images/mnist_01_y.npy
    modality: image
    max_samples: 400
    description: "MNIST binary (0 vs 1) -- 200 per class, 28x28
       greyscale"

filtrations:
  - name: vietoris_rips
    homology_dimensions: [0, 1]
  - name: weak_alpha
    homology_dimensions: [0, 1]
  - name: sparse_rips
    homology_dimensions: [0, 1]
    epsilon: 0.3
  - name: cubical
    homology_dimensions: [0, 1]

vectorizations:
  - name: persistence_image
    sigma: 0.1
    n_bins: 20
  - name: persistence_landscape
    n_layers: 3
    n_bins: 50
  - name: betti_curve
    n_bins: 50
  - name: silhouette
    n_bins: 50
  - name: persistence_entropy
    normalize: true
  - name: amplitude
    metric: bottleneck
  - name: persistence_statistics

classifiers:
  - name: svm_rbf
  - name: random_forest
  - name: logistic
  - name: svm_linear

evaluation:
  cv_folds: 5
  scoring: accuracy
  random_seed: 42
```

```
  repetitions: 1

output:
  db_path: data/tda/expanded_results.db
  log_level: WARNING
```

Appendix B

SQLite Schema Reference

```
CREATE TABLE IF NOT EXISTS runs (
    run_id          INTEGER PRIMARY KEY AUTOINCREMENT,
    dataset         TEXT    NOT NULL,
    filtration      TEXT    NOT NULL,
    vectorizer      TEXT    NOT NULL,
    classifier      TEXT    NOT NULL,
    repetition      INTEGER NOT NULL,
    started_at      TEXT,
    finished_at     TEXT,
    wall_time_s     REAL,
    peak_memory_mb  REAL
);

CREATE TABLE IF NOT EXISTS fold_results (
    fold_id     INTEGER PRIMARY KEY AUTOINCREMENT,
    run_id      INTEGER NOT NULL REFERENCES runs(run_id),
    fold        INTEGER NOT NULL,
    accuracy    REAL,
    f1          REAL,
    precision   REAL,
    recall      REAL
);

CREATE TABLE IF NOT EXISTS run_metadata (
    run_id          INTEGER PRIMARY KEY REFERENCES runs(run_id),
    pipeline_params TEXT,
    n_train         INTEGER,
```

```
    n_test          INTEGER,
    n_features      INTEGER
);

CREATE TABLE IF NOT EXISTS config_snapshot (
    id          INTEGER PRIMARY KEY CHECK (id = 1),
    yaml_text   TEXT    NOT NULL,
    saved_at    TEXT    NOT NULL
);

CREATE INDEX IF NOT EXISTS idx_runs_dataset ON runs(dataset);
CREATE INDEX IF NOT EXISTS idx_runs_filtration ON runs(filtration);
CREATE INDEX IF NOT EXISTS idx_fold_run ON fold_results(run_id);
```

Appendix C

Code Implementation

The factory classes and runner are available at `scripts/tda_benchmark/`. Key excerpts follow.

FiltrationFactory

```python
class FiltrationFactory:
    @staticmethod
    def create(name: str, **kwargs):
        mapping = {
            "vietoris_rips": VietorisRipsPersistence,
            "sparse_rips": SparseRipsPersistence,
            "cubical": CubicalPersistence,
            "weak_alpha": WeakAlphaPersistence,
        }
        cls = mapping.get(name)
        if cls is None:
            raise ValueError(f"Unknown filtration: {name}")
        return cls(n_jobs=1, **kwargs)
```

Listing C.1: FiltrationFactory.

VectorizationFactory (auto-flatten)

```python
if name != "persistence_statistics":
    return Pipeline([
        ("vec", vec),
        ("flatten", FunctionTransformer(
            lambda x: x.reshape(x.shape[0], -1),
            validate=False)),
    ])
return vec
```

Listing C.2: VectorizationFactory auto-flatten.

Execution Loop

```
for ds, fil, vec, clf in product(datasets, filtrations,
                                 vectorizers, classifiers):
    pipeline = Pipeline([
        ("filtration",  FiltrationFactory.create(fil.name, ...)),
        ("vectorizer",  VectorizationFactory.create(vec.name, ...)),
        ("classifier",  ClassifierFactory.create(clf.name, ...)),
    ])
    scores = cross_validate(pipeline, X, y,
        cv=StratifiedKFold(n_splits=cv_folds, shuffle=True),
        scoring=["accuracy", "f1_weighted",
                 "precision_weighted", "recall_weighted"])
```

Listing C.3: PipelineRunner execution loop.

PersistenceStatistics

```
class PersistenceStatistics(BaseEstimator, TransformerMixin):
    def transform(self, X):
        features = []
        for d in sorted(dims):
            mask = X[:, :, 2] == d
            births, deaths = X[:,:,0][mask], X[:,:,1][mask]
            lifespans = deaths - births
            for arr in [births, deaths, lifespans]:
                features.extend([np.nanmean(arr), np.nanstd(arr),
                                 np.nanmin(arr), np.nanmax(arr)])
        return np.column_stack(features)
```

Listing C.4: PersistenceStatistics transformer.

Appendix D

Extended Parameter Sweeps

The following sensitivity tests were conducted on reduced configuration subsets.

> **Provenance note.** The Takens sensitivity table below is an *illustrative reduced-sweep result* from an early exploratory phase: it was produced by a script that is no longer part of the verified pipeline, and no database producer for it exists in the repository. It is not part of the verified 616-configuration sweep, and its numbers should be treated as indicative only. The stage-impact claims of Chapter 4 rest on the repeated-CV analysis of §4.1 and the η^2 decomposition of Table 4.3, which are fully reproducible.

Takens Embedding Sensitivity

The claim that vectorization dominates on ECG200 was tested under alternative embedding parameters on a reduced sweep (2 filtrations × 2 vectorizers × 2 classifiers):

Table D.1: Stage-impact hierarchy under alternative (d, τ).

(d, τ)	Fil. Range	Vec. Range	Clf. Range
(2, 1)	0.09pp	1.72pp	1.14pp
(3, 1)	0.12pp	1.87pp	1.25pp
(5, 1)	0.15pp	1.94pp	1.31pp

Vectorization consistently dominates across all tested (d, τ) pairs. The filtration range remains below 0.15pp in all cases.

Framework Extensibility

The framework supports 7 filtrations (VR, Alpha, Sparse Rips, Čech, Cubical, Weighted Rips, Flagser), 11 vectorizers (PI, PL, Betti, Silhouette, Entropy, Amplitude, Heat Kernel, Complex Polynomials, Pairwise Distance, Number of Points, Persistence Statis-

tics), and 4 classifiers (SVM-linear, SVM-RBF, RF, Logistic). The 616-configuration sweep reported in Chapter 4 exercises 4, 7, and 4 of these respectively. Adding a new method to the sweep requires adding one entry to the YAML configuration (Appendix A). Adding a fundamentally new component type (e.g., a learned vectorizer) requires a new factory entry in `factories.py`.

Appendix E

Use of AI Tools

AI tools were used in the following capacities:

- **Code generation and debugging.** An AI coding assistant assisted with implementing the benchmarking framework: factory-pattern architecture, YAML loader, SQLite storage, pipeline runner, and analysis module. All code was reviewed, tested, and modified by me.
- **Literature synthesis.** AI tools assisted in extracting key findings from the prior-art survey.
- **Document preparation.** The structural template was adapted from previous work with AI assistance.

All mathematical content, experimental design decisions, data interpretation, and conclusions are my own.

Bibliography

[1] H. Adams, T. Emerson, M. Kirby, R. Neville, C. Peterson, P. Shipman, S. Chepushtanova, E. Hanson, F. Motta, and L. Ziegelmeier. Persistence images: A stable vector representation of persistent homology. *Journal of Machine Learning Research*, 18(8):1–35, 2017.

[2] D. Ali, A. Asaad, M.-J. Jimenez, V. Nanda, E. Paluzo-Hidalgo, and M. Soriano-Trigueros. A survey of vectorization methods for persistent homology. *IEEE Trans. Pattern Analysis and Machine Intelligence*, 45(12):14567–14588, 2023.

[3] D. Barnes, L. Polanco, and J. A. Perea. A comparative study of machine learning methods for persistence diagrams. *Frontiers in Artificial Intelligence*, 4:1–16, 2021.

[4] P. Bubenik. Statistical topological data analysis using persistence landscapes. *Journal of Machine Learning Research*, 16(3):77–102, 2015.

[5] M. Carrière, F. Chazal, Y. Ike, T. Lacombe, M. Royer, and Y. Umeda. PersLay: A neural network layer for persistence diagrams and new graph topological signatures. *Proceedings of AISTATS*, 2020.

[6] F. Chazal, B. T. Fasy, F. Lecci, A. Rinaldo, and L. Wasserman. Stochastic convergence of persistence landscapes and silhouettes. *Proceedings of the 30th Annual Symposium on Computational Geometry (SoCG)*, pp. 474–483, 2014.

[7] Y.-M. Chung and A. Lawson. Persistence curves: A canonical framework for summarizing persistence diagrams. *Journal of Machine Learning Research*, 23:1–62, 2022.

[8] D. Cohen-Steiner, H. Edelsbrunner, and J. Harer. Stability of persistence diagrams. *Discrete & Computational Geometry*, 37(1):103–120, 2007.

[9] F. Conti, D. Moroni, and M. A. Pascali. A topological machine learning pipeline for classification. *Mathematics* (MDPI), 10(17):3086, 2022. arXiv:2309.15276.

[10] F. Graf, P. Pellizzoni, M. Uray, S. Huber, and R. Kwitt. The flood complex: Large-scale persistent homology on millions of points. *Advances in Neural Information Processing Systems*, 2025. arXiv:2509.22432.

[11] C. Hofer, R. Kwitt, M. Niethammer, and A. Uhl. Deep learning with topological signatures. *Advances in Neural Information Processing Systems*, 2017.

[12] J. Leray. Sur la forme des espaces topologiques et sur les points fixes des représentations. *Journal de Mathématiques Pures et Appliquées*, 24:95–167, 1945.

[13] J. A. Perea, E. Munch, and F. Khasawneh. Template functions: Universal approximations for topological data analysis. *Foundations of Computational Mathematics*, 2022.

[14] M. Rucco, F. Castiglione, E. Merelli, and M. Pettini. Characterisation of the idiotypic immune network through persistent entropy. *Proceedings of ECCS 2014*, Springer Proceedings in Complexity, pp. 117–128, 2016.

[15] E. Somasundaram, S. Brown, A. Litzler, and R. Green. Benchmarking R packages for persistent homology computation. *Journal of Statistical Software*, 2021.

[16] D. Sulowska. Comparative evaluation of persistence diagram vectorisation methods in classification tasks. *Advances in Science and Technology Research Journal*, 20(7):54–70, 2026. DOI 10.12913/22998624/218660.

[17] F. Takens. Detecting strange attractors in turbulence. In *Dynamical Systems and Turbulence*, Lecture Notes in Mathematics 898, pp. 366–381. Springer, 1981.

[18] G. Tauzin, U. Lupo, L. Tunstall, J. B. Pérez, M. Caorsi, A. Medina-Mardones, A. Dassatti, and K. Hess. giotto-tda: A topological data analysis toolkit for machine learning. *Journal of Machine Learning Research*, 22(39):1–6, 2021.

[19] L. Telyatnikov, G. Bernardez, M. Montagna, M. Hajij, S. Carrasco, M. Vasylenko, et al. TopoBench: A framework for benchmarking topological deep learning. *Advances in Neural Information Processing Systems*, Datasets and Benchmarks Track, 2024. arXiv:2406.06642.

[20] R. Turkes, G. Montúfar, and N. Otter. On the effectiveness of persistent homology. *Advances in Neural Information Processing Systems*, 2022.

[21] H. A. Dau, A. Bagnall, K. Kamgar, C.-C. M. Yeh, Y. Zhu, S. Gharghabi, C. A. Ratanamahatana, and E. Keogh. The UCR time series archive. *IEEE/CAA Journal of Automatica Sinica*, 6(6):1293–1305, 2019.

[22] Y. LeCun, L. Bottou, Y. Bengio, and P. Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.

[23] F. Chazal, V. de Silva, and S. Oudot. Persistence stability for geometric complexes. *Geometriae Dedicata*, 178:295–331, 2015. arXiv:1207.3885.

[24] M. Hensel, M. Moor, and B. Rieck. A survey of topological machine learning methods. *Frontiers in Artificial Intelligence*, 4:681108, 2021.